

Shell and its scripting

Shell Programming

- Shell sits between the user and the UNIX system
- A shell script is a computer program that runs on the Unix shell which could be one of the following:
 - The Bourne family comprising the Bourne shell (/bin/sh) and its derivatives—the Korn shell (/bin/ksh) and Bash (/bin/bash).
 - The C Shell (/bin/csh) and its derivative, Tcsh (/bin/tcsh).

To know the shell you are using, invoke the command `echo $SHELL`.

Korn and Bash are supersets of Bourne, so anything that applies to Bourne also applies to them.

- A shell is a command-line interpreter and typical operations performed by shell scripts include file manipulation, program execution, and printing text.
- Shell scripts have several required constructs that tell the shell environment what to do and when to do it.
- The shell is a real programming language, with different variables, control structures, and so forth. No matter how complicated a script gets, it is still just a list of commands executed sequentially.

Tasks performed by the shell

It issues the prompt and waits for you to enter a command.

After a command is entered, the shell scans the command line for metacharacters and expands abbreviations (like the * in `rm *`) to re-create a simplified command line.

It then passes on the command line to the kernel for execution.

The shell waits for the command to complete and normally can't do any work while the command is running.

After command execution is complete, the prompt reappears and the shell returns to its waiting role to start the next cycle. You are now free to enter another command.

The Shell's Wild Cards and Application

<i>Wild Card</i>	<i>Matches</i>
<code>*</code>	Any number of characters including none
<code>?</code>	A single character
<code>[i j k]</code>	A single character—either an i, j, or k
<code>[x-z]</code>	A single character that is within the ASCII range of the characters x and z
<code>[!i j k]</code>	A single character that is not an i, j, or k (Not in C shell)
<code>[!x-z]</code>	A single character that is not within the ASCII range of the characters x and z (Not in C shell)
<code>{pat1, pat2...}</code>	<i>pat1, pat2, etc. (Not in Bourne Shell; see Going Further)</i>
<code>!(fname)</code>	All except <i>fname</i> (Korn and Bash; see Going Further)
<code>!(fname1 fname2)</code>	All except <i>fname1</i> and <i>fname2</i> (Korn and Bash; see Going Further)

The Shell's Wild Cards and Application

Examples

Command	Significance
<code>ls *.lst</code>	Lists all files with extension .lst
<code>mv * ../bin</code>	Moves all files to bin subdirectory of parent directory
<code>gzip .?*.*</code>	Compresses all files beginning with a dot, followed by one or more characters, then a second dot followed by one or more characters
<code>cp chap chap*</code>	Copies file chap to file chap* (* loses meaning here)
<code>cp ?????? progs</code>	Copies to progs directory all six-character filenames
<code>cmp rep[12]</code>	Compares files rep1 and rep2
<code>rm note[0-1][0-9]</code>	Removes files note00, note01 . . . through note19
<code>lp *.!o</code>	Prints all files having extensions except C object files
<code>cp ?*.*[!1238] ..</code>	Copies to the parent directory files having extensions with at least one character before the dot, but not having 1, 2, 3, or 8 as the last character

Special cases

Occurrence of a file names

chap*

\$ ls chap*

chap chap* chap01 chap02 chap03 chap04 chap15 chap16 chap17 chapx
chapy chapz

If we want to remove chap * only, it will remove all the file.

Escaping and Quoting

Escaping—Providing a \ (backslash) before the wild card to remove (escape) its special meaning.

`rm chap\*` : deleted `chap *` but Doesn't remove `chap1`, `chap2`

`rm chap0\[1-3\]` Removes only `chap0[1-3]`

`rm My\ Document.doc` (to escape the space in the filename)

Without the \ `rm` would see two files

`$ echo \ : \`

`$ echo The newline character is \n The newline character is \n`

Quoting—

When a command argument is enclosed in quotes, the meanings of all enclosed special characters are turned off. Or in other words Anything within these quotes (barring few exceptions) is left alone by the shell and not interpreted

```
echo \'
```

Displays a \

```
rm 'chap*'
```

Removes file chap*

```
rm "My Document.doc"
```

Removes file My Document.doc

```
$ echo 'The characters |, and $ are also special'
```

The characters |, and \$ are also special

```
$ type echo
```

Redirection

A stream is simply a sequence of bytes. When a user logs in, the shell makes available three files representing three streams. Each stream is associated with a default device, and—generically speaking—this device is the terminal:

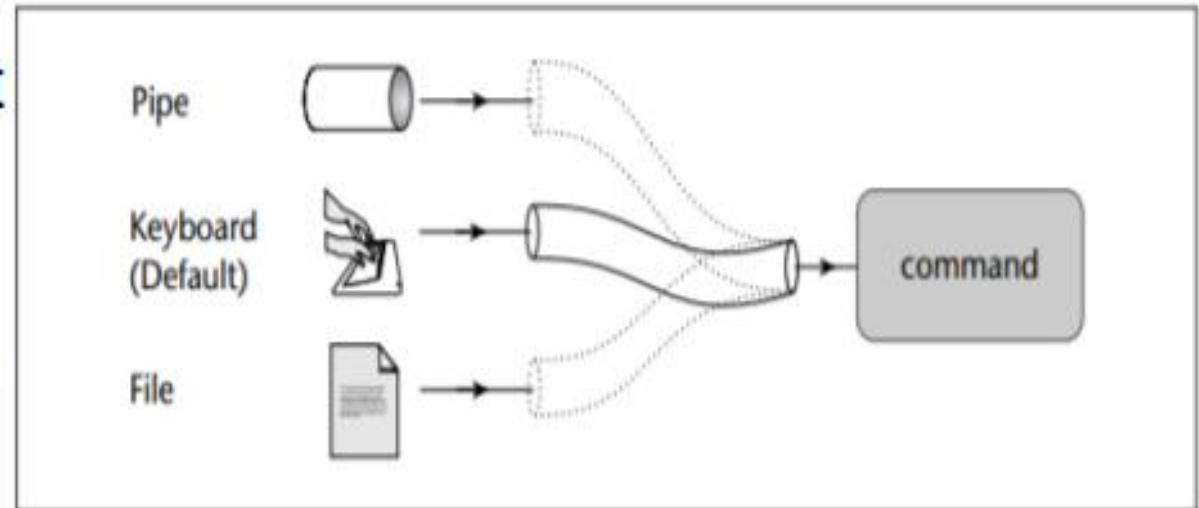
Standard input—The file (or stream) representing input, which is connected to the keyboard.

Standard output—The file (or stream) representing output, which is connected to the display.

Standard error—The file (or stream) representing error messages that emanate from the command or shell. This is also connected to the display.

Standard Input

Commands like `cat` and `wc` help to read disk files. They have an additional method of taking input. When they are used without arguments, they read the file representing the standard input. The keyboard, the default source. A file using redirection with the `<` symbol (a metacharacter). Another program using a pipeline (to be taken up later).



1. On seeing the <, the shell opens the disk file, /etc/passwd, for reading.
2. It unplugs the standard input file from its default source and assigns it to /etc/passwd.
3. wc reads from standard input that has previously been reassigned by the shell to /etc/passwd.

```
hvr@hvr-HP-Pavilion-Notebook:~$ wc </etc/passwd
 47   82 2789
hvr@hvr-HP-Pavilion-Notebook:~$ wc /etc/passwd
 47   82 2789 /etc/passwd
hvr@hvr-HP-Pavilion-Notebook:~$ wc </etc/passwd
 47   82 2789
```

Taking Input Both from File and Standard Input

When a command takes input from multiple sources—say, a file and standard input—the `-` symbol must be used to indicate the sequence of taking input. The meaning of the following sequences should be quite obvious:

`cat foo -` - First from `foo`, then standard input,

```
hvr@hvr-HP-Pavilion-Notebook:~$ cat foo
date
hvr@hvr-HP-Pavilion-Notebook:~$ cat foo -
date
apple
apple
apple
apple
```

Standard Output

All commands displaying output on the terminal actually write to the standard output file as a stream of characters, and not directly to the terminal as such. There are three possible destinations for this stream

The terminal, the default destination.

A file, using the redirection symbols `>` and `>>`.

As input to another program using a pipeline (to be taken up later).

```
$ wc /etc/passwd > newfile
$ cat newfile
      21      45    1083 /etc/passwd
```

On seeing the `>`, the shell opens the disk file, `newfile`, for writing.

2. It unplugs the standard output file from its default destination and assigns it to `newfile`.

3. `wc` (and not the shell) opens the file `/etc/passwd` for reading.

4. `wc` writes to standard output, which was previously reassigned by the shell to `newfile`.

Pipe (|)

- The | (pipe) connects input and output streams together so that one command takes input from the other.
- You know the who command produces a list of users, one user per line. Let's use redirection to save this output in a file:

```
$ who > user.txt
$ cat user.txt
root      console      Aug  1 07:51    (:0)
romeo     pts/10        Aug  1 07:56    (pc123.heavens.com)
juliet    pts/6         Aug  1 02:10    (pc125.heavens.com)
project   pts/8         Aug  1 02:16    (pc125.heavens.com)
andrew    pts/14        Aug  1 08:36    (mercury.heavens.com)
```

- Let's now use wc to make a complete count of this file's contents. We'll use the -l option to count the number of lines only, and redirect wc's input so that the filename doesn't appear in the output:

```
$ wc -l < user.txt
 5
```

Counts the number of users

Pipe (|)

- Using an intermediate file (user.txt), we effectively counted the number of users. This method of running two commands separately has two obvious disadvantages:
 1. For long-running commands, this process can be slow. The second command can't act unless the first has completed its job.
 2. You require an intermediate file that has to be removed after completion of the job. When handling large files, temporary files can build up easily and eat up disk space quickly.
- The shell can connect these streams using a special operator—the | (pipe) and avoid the creation of the disk file. You can make *who* and *wc* work in tandem so that one takes input from the other:

```
$ who | wc -l  
5
```

No intermediate files created

```
$ ls | wc -l  
15
```

```
ls | wc -l > fkount
```

Pipe (|)

- There's no restriction on the number of commands you can use in a pipeline. But you must know the behavioral properties of these commands to place them there.
- let's use the wc command to display the total size of all C programs:

```
$ wc -c *.c
 2078 backtalk.c
  231 beyond_array.c
  ....
 1944 dog.c
   884 hexdump.c
   214 swap.c
940101 total
```

Output trimmed here

- what we need is a single figure representing the total size.

```
$ cat *.c | wc -c
940101
```

tee: CREATING A TEE

- tee is an external command and not a feature of the shell. It handles a character stream by splitting its input into two components. It saves one component in a file and writes the other to the standard output.
- Being also a filter (which uses standard input and standard output), tee can be placed anywhere in a pipeline.
- tee doesn't perform any filtering action on its input; it gives out exactly what it takes.
- The following command sequence uses tee to display the output of who and saves this output in a file as well:

```
$ who | tee user.txt
```

romeo	pts/2	Sep 7 08:41	(pc123.heavens.com)
juliet	pts/3	Sep 7 17:58	(pc122.heavens.com)
sumit	pts/5	Sep 7 18:01	(mercury.heavens.com)

tee: CREATING A TEE

- Since *cmp* compares two files and one of them can be represented by the standard input file, we can arrange to provide this input from the output of who:

```
$ who | cmp user.txt -  
$ _
```

Second file is standard input

- To display both the list of users and its count on the terminal, you can use the device name */dev/tty* as an argument to tee:

```
$ who | tee /dev/tty | wc -l  
romeo      pts/2  
juliet     pts/3  
sumit      pts/5  
          3
```

/dev/tty used as command argument

```
Sep  7 08:41 (pc123.heavens.com)  
Sep  7 17:58 (pc122.heavens.com)  
Sep  7 18:01 (mercury.heavens.com)
```

It's like running who | wc -l

What is Shell?

- A **Shell** provides an interface to the Unix system. It gathers input from users and executes programs based on that input. When a program finishes executing, it displays that program's output.
- Shell is an environment in which you can run your commands, programs, and shell scripts. There are different flavors of a shell, just as there are different flavors of operating systems.
- **Shell Prompt:** The prompt, \$, which is called the command prompt, is issued by the shell. While the prompt is displayed, you can type a command.
- Shell reads your input after you press Enter. It determines the command you want executed by looking at the first word of your input. A word is an unbroken set of characters. Spaces and tabs separate words.

Simple Shell Example

- The following script uses the **read** command which takes the input from the keyboard and assigns it as the value of the variable PERSON and finally prints it on STDOUT.

```
#!/bin/sh
# Script follows here:
echo "Favorite color?"
read color
echo " Favorite color, $ color "
```

- Here is a **sample run of the script** –

```
./test.sh
Favorite color?
pink
Favorite color,pink
$
```

Shell Script

- The basic concept of a shell script is a list of commands, which are listed in the order of execution. A good shell script will have comments, preceded by # sign, describing the steps.
- **Example Script:** Assume we create a test.sh script. All the scripts would have the .sh extension. Before you add anything else to your script, you need to alert the system that a shell script is being started. This is done using the shebang construct. For example –
`#!/bin/sh`
- This tells the system that the commands that follow are to be executed by the Bourne shell. It's called a shebang because the # symbol is called a hash, and the ! symbol is called a bang.
- To create a script containing these commands, you put the shebang line first and then add the commands –
`#!/bin/bash`
`pwd`
`ls`

Running Shell Script

1. You can write comments in the script as follows –

```
#!/bin/bash  
# Author : hvr  
pwd  
ls
```

second.sh

2. Save the above content and make the script executable –

```
$chmod +x test.sh
```

3. The shell script is now ready to be executed –

```
$/test.sh
```

4. Upon execution, you will receive the following result –

```
/home/hvr  
index.htm unix-basic_utilities.htm  unix-directories.htm  
test.sh    unix-communication.htm  unix-environment.htm
```

- Note – To execute a program available in the current directory, use `./program_name`

```
hvr@hvr-HP-Pavilion-Notebook:~$ vi testShell2.sh
hvr@hvr-HP-Pavilion-Notebook:~$ chmod u+x testShell12.sh
chmod: cannot access 'testShell12.sh': No such file or directory
hvr@hvr-HP-Pavilion-Notebook:~$ chmod u+x testShell2.sh
hvr@hvr-HP-Pavilion-Notebook:~$ ./testShell2.sh
/home/hvr
'$'
apple23.txt
apple2.txt
apple.txt
archive1.zip
archive.zip
'cad materiala'
Desktop
Documents
Downloads
ECED
file1
first
foo
fruits
mtech
Music
outputFile.txt
Pictures
progs
Public
scripts
snap
statements.sh
Templates
test1.pl
test1.sh
testDoWhile.pl
testElif.pl
testFile
testFile1
testFile1.pl
testFile2
testFile2.txt
testFile3
testFile4
testFile.txt
testFormat1.pl
testFormat.pl
testFor.pl
testgoto.pl
testIf.pl
test.sh
testShell2.sh
testShell.sh
testUntil.pl
testVI
testViEditor.txt
testWhile.pl
testWHile.pl
textFile
typescript
unixFiles
Videos
```

Extended Shell Scripts

- The following script uses the read command which takes the input from the keyboard and assigns it as the value of the variable PERSON and finally prints it on STDOUT.

```
hvr@hvr-HP-Pavilion-Notebook:~$ vi testShell3.sh
hvr@hvr-HP-Pavilion-Notebook:~$ chmod u+x testShell3.sh
hvr@hvr-HP-Pavilion-Notebook:~$ ./testShell3.sh
What is your name?
hvr
Hello, hvr
```

```
#!/bin/sh
# Author : hvr
echo "What is your name?"
read name
echo "Hello, $name"
```

Using Shell Variables

- A variable is a character string to which we assign a value. The value assigned could be a number, text, filename, device, or any other type of data.
- A variable is nothing more than a pointer to the actual data. The shell enables you to create, assign, and delete variables.
- The name of a variable can contain only letters (a to z or A to Z), numbers (0 to 9) or the underscore character (_).
- By convention, Unix shell variables will have their names in UPPERCASE.
- The following examples are **valid variable names** –
_ALI TOKEN_A VAR_1 VAR_2
- Following are the examples of **invalid variable names** –
2_VAR
-VARIABLE VAR1-VAR2 VAR_A!
- The reason you cannot use other characters such as !, *, or - is that these characters have a special meaning for the shell.

Defining Variables

- Variables are defined as follows

`variable_name=variable_value`

- For example

`NAME="Akshit"`

- The example defines a variable NAME and assigns the value "Zara Ali" to it. Variables of this type are called scalar variables (can hold only one value at a time).

- Shell enables you to store any value you want in a variable. For example

`VAR1="akshit" VARnew=100`

- **To access the value stored in a variable, prefix its name with the dollar sign (\$):** For example, the following script will access the value of defined variable NAME and print it on STDOUT –

`#!/bin/sh`

`NAME="akshit"`

`echo $NAME`

- The above script will produce the following value –

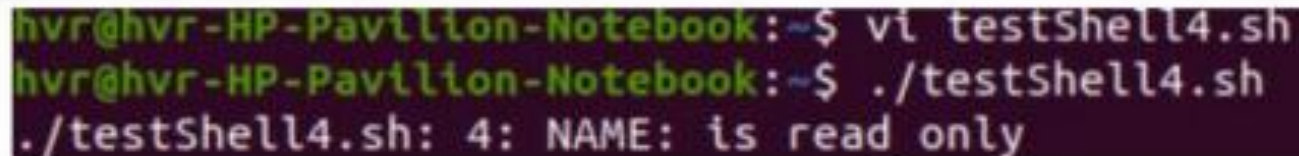
`akshit`

Defining Variables

- Shell provides a way to mark variables as read-only by using the **read-only command**. After a variable is marked read-only, its value cannot be changed. For example,

```
#!/bin/sh  
NAME="akshit"  
readonly NAME  
NAME="akshit"
```

- The above script will generate the following result –
/bin/sh: NAME: This variable is read only.



```
hvr@hvr-HP-Pavillion-Notebook:~$ vi testShell4.sh  
hvr@hvr-HP-Pavillion-Notebook:~$ ./testShell4.sh  
./testShell4.sh: 4: NAME: is read only
```

- Unsetting or deleting a variable directs the shell to remove the variable from the list of variables that it tracks. Once you unset a variable, you cannot access the stored value in the variable. Following is the syntax to unset a defined variable
unset variable_name
- The above command unsets the value of a defined variable. A simple example
#!/bin/sh NAME="Zara Ali" unset NAME echo \$NAME
- The above example does not print anything. You cannot use the unset command to unset variables that are marked read-only.

Variable Types

- When a shell is running, three main types of variables are present –
 - **Local Variables** – A local variable is a variable that is present within the current instance of the shell. It is not available to programs that are started by the shell. They are set at the command prompt.
 - **Environment Variables** – An environment variable is available to any child process of the shell. Some programs need environment variables in order to function correctly. Usually, a shell script defines only those environment variables that are needed by the programs that it runs.
 - **Shell Variables** – A shell variable is a special variable that is set by the shell and is required by the shell in order to function correctly. Some of these variables are environment variables whereas others are local variables.

Special Variable

- The nonalphanumeric characters are used in for creating special variables.
- For example, \$ character represents the process ID number, or PID, of the current shell.

\$0	The filename of the current script.
\$n	These variables correspond to the arguments with which a script was invoked. Here n is a positive decimal number corresponding to the position of an argument (the first argument is \$1, the second argument is \$2, and so on).
\$#	The number of arguments supplied to a script.
\$*	All the arguments are double quoted. If a script receives two arguments, \$* is equivalent to \$1 \$2.
\$@	All the arguments are individually double quoted. If a script receives two arguments, @\$ is equivalent to \$1 \$2.
\$?	The exit status of the last command executed.
\$\$	The process number of the current shell. For shell scripts, this is the process ID under which they are executing.
\$_	The process number of the last background command.

Command-Line Arguments

- The command-line arguments \$1, \$2, \$3, ...\$9 are positional parameters, with \$0 pointing to the actual command, program, shell script, or function and \$1, \$2, \$3, ...\$9 as the arguments to the command.

- Following script uses various special variables related to the command line –

```
#!/bin/sh
echo "File Name: $0"
echo "First Parameter : $1"
echo "Second Parameter : $2"
echo "Quoted Values: $@"
echo "Quoted Values: $*"
echo "Total Number of Parameters : $#"
```

- Here is a sample run for the above script –

```
./test.sh Zara Ali
File Name : ./test.sh
First Parameter : Zara
Second Parameter : Ali
Quoted Values: Zara Ali
Quoted Values: Zara Ali
Total Number of Parameters : 2
```

fourth.sh

Special Parameters \$* and \$@

- There are special parameters that allow accessing all the command-line arguments at once. \$* and \$@ both will act the same unless they are enclosed in double quotes.
- **Both the parameters specify the command-line arguments.** However, the "\$*" special parameter takes the entire list as one argument with spaces between and the "\$@" special parameter takes the entire list and separates it into separate arguments.
- We can write the shell script as shown below to process an unknown number of commandline arguments with either the \$* or \$@ special parameters –

```
#!/bin/sh
for TOKEN in $* do
    echo $TOKEN done
```

five.sh

- Here is a sample run for the above script –
\$./test.sh Zara Ali 10 Years Old Zara
Ali
10
Years Old
- Note – Here **do...done** is a kind of loop

Exit Status

- The `$?` variable represents the exit status of the previous command.
- Exit status is a numerical value returned by every command upon its completion. As a rule, most commands return an exit status of 0 if they were successful, and 1 if they were unsuccessful.
- Some commands return additional exit statuses for particular reasons. For example, some commands differentiate between kinds of errors and will return various exit values depending on the specific type of failure.
- Following is the example of successful command –
\$./test.sh Zara Ali
File Name : ./test.sh
First Parameter : Zara
Second Parameter : Ali
Quoted Values: Zara Ali
Quoted Values: Zara Ali
Total Number of Parameters : 2
\$echo \$?
0
\$

Array Variable

- **Array variable** can hold multiple values at the same time. Arrays provide a method of grouping a set of variables.
- Suppose you are trying to represent the names of various students as a set of variables. Each of the individual variables is a scalar variable as follows –
`NAME01="Zara" NAME02="Qadir"`
`NAME03="Mahnaz" NAME04="Ayan" NAME05="Daisy"`
- We can use a single array to store all the above mentioned names. Following is the simplest method of creating an array variable. This helps assign a value to one of its indices.
`array_name[index]=value`
- Here `array_name` is the name of the array, `index` is the index of the item in the array that you want to set, and `value` is the value you want to set for that item. An example
`NAME[0]="Zara" NAME[1]="Qadir"`
`NAME[2]="Mahnaz" NAME[3]="Ayan" NAME[4]="Daisy"`
- If you are using the `ksh` shell, here is the syntax of array initialization –
`set -A array_name value1 value2 ... valuen`
- If you are using the `bash` shell, here is the syntax of array initialization –
`array_name=(value1 ... valuen)`

Accessing Array Values

- Once you have set any array variable, you access it as follows:
`${array_name[index]}`
- Here `array_name` is the name of the array, and `index` is the index of the value to be accessed. Example,

```
#!/bin/sh NAME[0]="Zara" NAME[1]="Qadir"  
NAME[2]="Mahnaz"  
echo "First Index: ${NAME[0]}" echo "Second Index: ${NAME[1]}"
```
- The above example will generate the following result –

```
./test.sh  
First Index: Zara Second Index: Qadir
```
- You can access all the items in an array in one of the following ways –
`${array_name[*]}`
`${array_name[@]}`
- Here `array_name` is the name of the array you are interested in. For example

```
#!/bin/sh NAME[0]="Zara" NAME[1]="Qadir"  
NAME[2]="Mahnaz"  
echo "First Method: ${NAME[*]}"  
echo "Second Method: ${NAME[@]}"
```
- The above example will generate the following result –

```
./test.sh  
First Method: Zara Qadir Mahnaz Second Method: Zara Qadir Mahnaz
```

Shell Basic Operators

- There are various operators supported by each shell. We will discuss about Bourne shell. Following operators are frequently used
 - Arithmetic Operators
 - Relational Operators
 - Boolean Operators
 - String Operators
 - File Test Operators
- Bourne shell didn't originally have any mechanism to perform simple arithmetic operations but it uses external programs, either **awk** or **expr**.
- The following example shows how to add two numbers –

```
#!/bin/sh  
val=`expr 2 + 2`  
echo "Total value : $val"
```
- The above script will generate the following result:
Total value : 4
- The following points need to be considered while adding:
 - There must be spaces between operators and expressions. For example, 2+2 is not correct; it should be written as 2 + 2.
 - The complete expression should be enclosed between ' ` ', called the backtick.

Arithmetic Operators

- The following arithmetic operators are supported by Bourne Shell. Assume variable **a** holds 10 and variable **b** holds 20.
- All the conditional expressions should be inside square braces with spaces around them, for example [**\$a == \$b**] is correct while, [**\$a==\$b**] is incorrect.
- All the arithmetical calculations are done using long integers.

Operator	Description	Example
+ (Addition)	Adds values on either side of the operator	`expr \$a + \$b` will give 30
- (Subtraction)	Subtracts right hand operand from left hand operand	`expr \$a - \$b` will give -10
* (Multiplication)	Multiplies values on either side of the operator	`expr \$a \* \$b` will give 200
/ (Division)	Divides left hand operand by right hand operand	`expr \$b / \$a` will give 2
% (Modulus)	Divides left hand operand by right hand operand and returns remainder	`expr \$b % \$a` will give 0
= (Assignment)	Assigns right operand in left operand	a = \$b would assign value of b into a

Relational Operators

- Bourne Shell supports the following relational operators that are specific to numeric values. **These operators do not work for string values unless their value is numeric.**
- For example, following operators will work to check a relation between 10 and 20 as well as in between "10" and "20" but not in between "ten" and "twenty".
- Assume variable **a** holds 10 and variable **b** holds 20.
- **All the conditional expressions should be placed inside square braces with spaces around them.** For example, [\$a <= \$b] is correct whereas, [\$a <= \$b] is incorrect.

Operator	Description	Example
-eq	Checks if the value of two operands are equal or not; if yes, then the condition becomes true.	[\$a -eq \$b] is not true.
-ne	Checks if the value of two operands are equal or not; if values are not equal, then the condition becomes true.	[\$a -ne \$b] is true.
-gt	Checks if the value of left operand is greater than the value of right operand; if yes, then the condition becomes true.	[\$a -gt \$b] is not true.
-lt	Checks if the value of left operand is less than the value of right operand; if yes, then the condition becomes true.	[\$a -lt \$b] is true.
-ge	Checks if the value of left operand is greater than or equal to the value of right operand; if yes, then the condition becomes true.	[\$a -ge \$b] is not true.

Boolean Operators

- The following Boolean operators are supported by the Bourne Shell.
- Assume variable **a** holds 10 and variable **b** holds 20

Operator	Description	Example
!	This is logical negation. This inverts a true condition into false and vice versa.	[! false] is true.
-o	This is logical OR . If one of the operands is true, then the condition becomes true.	[\$a -lt 20 -o \$b -gt 100] is true.
-a	This is logical AND . If both the operands are true, then the condition becomes true otherwise false.	[\$a -lt 20 -a \$b -gt 100] is false.

String Operators

- The following string operators are supported by Bourne Shell.
- Assume variable **a** holds "abc" and variable **b** holds "efg"

Operator	Description	Example
=	Checks if the value of two operands are equal or not; if yes, then the condition becomes true.	[\$a = \$b] is not true.
!=	Checks if the value of two operands are equal or not; if values are not equal then the condition becomes true.	[\$a != \$b] is true.
-z	Checks if the given string operand size is zero; if it is zero length, then it returns true.	[-z \$a] is not true.
-n	Checks if the given string operand size is non-zero; if it is nonzero length, then it returns true.	[-n \$a] is not false.

File Test Operators

We have a few operators that can be used to test various properties associated with a Unix file. Assume a variable **file** holds an existing file name "test" the size of which is 100 bytes and has **read**, **write** and **execute** permission on.

Operator	Description	Example
-b file	Checks if file is a block special file; if yes, then the condition becomes true.	[-b \$file] is false.
-c file	Checks if file is a character special file; if yes, then the condition becomes true.	[-c \$file] is false.
-d file	Checks if file is a directory; if yes, then the condition becomes true.	[-d \$file] is not true.
-f file	Checks if file is an ordinary file as opposed to a directory or special file; if yes, then the condition becomes true.	[-f \$file] is true.
-g file	Checks if file has its set group ID (SGID) bit set; if yes, then the condition becomes true.	[-g \$file] is false.
-k file	Checks if file has its sticky bit set; if yes, then the condition becomes true.	[-k \$file] is false.
-p file	Checks if file is a named pipe; if yes, then the condition becomes true.	[-p \$file] is false.
-t file	Checks if file descriptor is open and associated with a terminal; if yes, then the condition becomes true.	[-t \$file] is false.
-u file	Checks if file has its Set User ID (SUID) bit set; if yes, then the condition becomes true.	[-u \$file] is false.
-r file	Checks if file is readable; if yes, then the condition becomes true.	[-r \$file] is true.
-w file	Checks if file is writable; if yes, then the condition becomes true.	[-w \$file] is true.
-x file	Checks if file is executable; if yes, then the condition becomes true.	[-x \$file] is true.
-s file	Checks if file has size greater than 0; if yes, then condition becomes true.	[-s \$file] is true.

Shell Decision Making

- While writing a shell script, there may be a situation when user needs to adopt one path out of the given two paths.
- Therefore, conditional statements are used that allow program to make correct decisions and perform the right actions.
- Unix Shell supports conditional statements which are used to perform different actions based on different conditions.
- Two decision-making statements are –
 - The **if...else** statement
 - The **case...esac** statement

The if...else and case...esac statements

- If else statements are **useful decision-making statements** which can be used to select an option from a given set of options.
- Unix Shell supports following forms of **if...else** statement –
if...fi statement if...else...fi statement if...elif...else...fi statement
- Most of the if statements check relations using relational operators.
- You can use multiple **if...elif** statements to perform a multiway branch. However, this is not always the best solution, especially when all of the branches depend on the value of a single variable.
- Unix Shell supports **case...esac** statement which handles exactly this situation, and it does so more efficiently than repeated **if...elif** statements.
- There is only one form of **case...esac** statement
case...esac statement

Example: if...fi statement

- Syntax

```
if [ expression ] then  
    Statement(s) to be executed if expression is true  
fi
```

- Example

```
#!/bin/sh a=10 b=20  
if [ $a == $b ] then  
    echo "a is equal to b"  
fi  
if [ $a != $b ] then  
    echo "a is not equal to b"  
fi
```

- The above script will generate the following result –
a is not equal to b

Testifi.sh

Example: if...else...fi statement

- **Syntax**

```
if [ expression ] then  
    Statement(s) to be executed if expression is true  
else  
    Statement(s) to be executed if expression is not true  
fi
```

- **Example**

```
#!/bin/sh a=10 b=20  
if [ $a == $b ]  
then  
    echo "a is equal to b" else  
    echo "a is not equal to b"  
fi
```

testifelse.sh

- Upon execution, you will receive the following result –
a is not equal to b

Example: if...elif...fi statement

- **Syntax**

```
if [ expression 1 ] then
    Statement(s) to be executed if
    expression 1 is true
elif [ expression 2 ]
    then
    Statement(s) to be executed if
    expression 2 is true
elif [ expression 3 ]
    then
    Statement(s) to be executed if
    expression 3 is true
else
    Statement(s) to be executed if
    no
    expression is true
fi
```

- **Example**

```
#!/bin/sh a=10 b=20
if [ $a == $b ]
then
    echo "a is equal to b"
elif [ $a -gt $b ]
then
    echo "a is greater than b"
elif [ $a -lt $b ]
then
    echo "a is less than b"
else
    echo "None of the condition
    met"
fi
```

- Upon execution, you will receive the following result –
a is less than b

testifelif.sh

Example: case...esac statement

testcaseesac.sh

- **Syntax**

```
case word in pattern1)
    Statement(s) to be executed if
    pattern1 matches
;;
pattern2)
    Statement(s) to be executed if
    pattern2 matches
;;
pattern3)
    Statement(s) to be executed if
    pattern3 matches
;;
*)
    Default condition to be
    executed
;;
esac
```

- **Example**

```
#!/bin/sh FRUIT="kiwi" case
"$FRUIT" in
    "apple") echo "Apple pie is quite
    tasty."
;;
    "banana") echo "I like banana
    nut bread."
;;
    "kiwi") echo "New Zealand is
    famous for kiwi."
;;
esac
```

- **Upon execution, you will receive the following result –**
New Zealand is famous for kiwi.

Shell Loop Types

- A loop is a powerful programming tool that enables you to execute a set of commands repeatedly.
- Following types of loops available to shell programmers
 - The **while** loop
 - The **for** loop
 - The **until** loop
 - The **select** loop
- **You will use different loops based on the situation.** For example, the while loop executes the given commands until the given condition remains true; the until loop executes until a given condition becomes true.
- Once you have good programming practice you will gain the expertise in using appropriate loop based on the situation.

for Loops

- for loops allow the repetition of a command for a specific set of values
- Syntax:

```
for var in value1 value2 ...  
do  
    command_set  
done
```

- command_set is executed with each value of var (value1, value2, ...) in sequence

for Loop Example (1)

```
#!/bin/sh
# timestable – print out a multiplication table
for i in 1 2 3
do
    for j in 1 2 3
    do
        value=`expr $i \* $j`
        echo -n "$value "
    done
    echo
done
```

forloop1.sh

for Loop Example (2)

```
#!/bin/sh  
# loop over files  
files=`ls`  
for i in $files  
do  
    echo -n "$i "  
done
```

- Find filenames in files in the current directory

forloop2.sh

The while Loop

- While loops repeat statements as long as the next Unix command is successful.
- For example:

```
#!/bin/sh
i=1
sum=0
while [ $i -le 100 ]; do
    sum=`expr $sum + $i`
    i=`expr $i + 1`
done
echo The sum is $sum.
```

testwhileloop.sh

The until Loop

- Until loops repeat statements until the next Unix command is successful.
- For example:

```
#!/bin/sh
x=1
until [ $x -gt 3 ]; do
    echo x = $x
    x=`expr $x + 1`
done
```

testuntilloop.sh

The select Loop

- The select loop provides an easy way to create a numbered menu from which users can select options.
- Not available in sh.
- Syntax:

```
select var in word1 word2 ... wordN
```

```
do
```

```
    Statement(s) to be executed for every word.
```

```
done
```

The select Loop Example

```
#!/bin/bash
```

```
select DRINK in tea cofee water juice appe all none
do
    case $DRINK in
        tea|cofee|water|all)
            echo "Go to canteen"
            ;;
        juice|appe)
            echo "Available at home"
            ;;
        none)
            break
            ;;
        *) echo "ERROR: Invalid selection"
            ;;
    esac
done
```

testSelectLoop.sh

Nesting Loops

- All the loops support nesting concept which means you can put one loop inside another similar one or different loops. This nesting can go up to unlimited number of times based on your requirement.
- An example of nesting while loop. The other loops can be nested based on the programming requirement in a similar way –
- Syntax

```
while command1 ; # this is loop1, the outer loop do
    Statement(s) to be executed if command1 is true
while command2 ; # this is loop2, the inner loop do
    Statement(s) to be executed if command2 is true
done
Statement(s) to be executed if command1 is true
done
```

Nesting Loops Example

- A simple example of loop nesting.
- Let's add another countdown loop inside the loop that you used to count to nine
- This will produce the following result. It is important to note how echo -n works here.
- Here -n option lets echo avoid printing a new line character.

```
#!/bin/sh a=0
```

```
while [ "$a" -lt 10 ] # this is loop1
do
  b="$a"
  while [ "$b" -ge 0 ] # this is loop2
  do
    echo -n "$b " b=`expr $b - 1`
  done echo
  a=`expr $a + 1`
done
```

```
1 0
2 1 0
3 2 1 0
4 3 2 1 0
5 4 3 2 1 0
6 5 4 3 2 1 0
7 6 5 4 3 2 1 0
8 7 6 5 4 3 2 1 0
9 8 7 6 5 4 3 2 1 0
```

Shell Loop Control

- There are different control can be used to stop a loop or skip iterations of the loop.
 - The break statement
 - The continue statement
- All the loops have a limited life and they come out once the condition is false or true depending on the loop.
- A loop may continue forever if the required condition is not met. A loop that executes forever without terminating executes for an infinite number of times, such loops are called infinite loops.
- A simple example that uses the while loop to display the numbers zero to nine –

```
#!/bin/sh a=10
until [ $a -lt 10 ] do
    echo $a
    a=`expr $a + 1` done
```
- This loop continues forever because a is always greater than or equal to 10 and it is never less than 10.

The break Statement

- The break statement is used to terminate the execution of the entire loop, after completing the execution of all of the lines of code up to the break statement. It then steps down to the code following the end of the loop.
- Syntax: The following break statement is used to come out of a loop –
`break`
- The break command can also be used to exit from a nested loop using this format –
`break n`
- Here n specifies the n^{th} enclosing loop to the exit from.

The break Statement:

Example

- A simple example which shows that loop terminates as soon as a becomes 5

```
#!/bin/sh
a=0
while [ $a -lt 10 ] do
    echo $a
    if [ $a -eq 5 ]
    then
        break
    fi
    a=`expr $a + 1` done
```

- Upon execution, you will receive the following result –

```
0
1
2
3
4
5
```

The continue statement

- The continue statement is similar to the break command, except that it causes the current iteration of the loop to exit, rather than the entire loop.
- This statement is useful when an error has occurred but you want to try to execute the next iteration of the loop.
- Syntax
`continue`
- Like with the break statement, an integer argument can be given to the continue command to skip commands from nested loops.
`continue n`
- Here n specifies the n^{th} enclosing loop to continue from.

Example

- The following loop makes use of the continue statement which returns from the continue statement and starts processing the next statement

```
#!/bin/sh
NUMS="1 2 3 4 5 6 7"
for NUM in $NUMS do
    Q=`expr $NUM % 2` if [ $Q -eq 0 ]
    then
        echo "Number is an even
number!!"
        continue
    fi
    echo "Found odd number" done
```

- Upon execution, you will receive the following result –

```
Found odd number
Number is an even number!! Found
odd number
Number is an even number!! Found
odd number
Number is an even number!! Found
odd number
```

Shell Substitution: Escape Sequence

- The shell performs substitution when it encounters an expression that contains one or more special characters.
- Example, the printing value of the variable is substituted by its value. Same time, "\n" is substituted by a new line

```
#!/bin/sh a=10
echo -e "Value of a is $a \n"
```
- You will receive the following result. **The -e option enables the interpretation of backslash escapes.**
Value of a is 10
- Following is the result without -e option
Value of a is 10\n
- The following **escape sequences** which can be used in echo command

Escape	Description
\\	backslash
\a	alert (BEL)
\b	backspace
\c	suppress trailing newline
\f	form feed
\n	new line
\r	carriage return
\t	horizontal tab
\v	vertical tab

Command Substitution

- Command substitution is the mechanism by which the shell performs a given set of commands and then substitutes their output in the place of the commands.
- The command substitution is performed when a command is given as ``command``
- When performing the command substitution you use the backquote, not the single quote character.
- Example: Command substitution is generally used to assign the output of a command to a variable.

```
#!/bin/sh DATE=`date`  
echo "Date is $DATE"  
USERS=`who | wc -l`  
echo "Logged in user are $USERS"  
UP=`date ; uptime` echo "Uptime is $UP"
```
- Upon execution, you will receive the following result –
Date is Thu Jul 2 03:59:57 MST 2009
Logged in user are 1
Uptime is Thu Jul 2 03:59:57 MST 2009
03:59:57 up 20 days, 14:03, 1 user, load
avg: 0.13, 0.07, 0.15

Variable Substitution

- Variable substitution enables the shell programmer to manipulate the value of a variable based on its state.
- Here is the following table for all the possible substitutions.

Form	Description
<code>\${var}</code>	Substitute the value of <i>var</i> .
<code>\${var:-word}</code>	If <i>var</i> is null or unset, <i>word</i> is substituted for var . The value of <i>var</i> does not change.
<code>\${var:=word}</code>	If <i>var</i> is null or unset, <i>var</i> is set to the value of word .
<code>\${var:?message}</code>	If <i>var</i> is null or unset, <i>message</i> is printed to standard error. This checks that variables are set correctly.
<code>\${var:+word}</code>	If <i>var</i> is set, <i>word</i> is substituted for <i>var</i> . The value of <i>var</i> does not change.

Variable Substitution:

Example

- Following is the example to show various states of the above substitution
 - `#!/bin/sh`
 - `echo ${var:-"Variable is not set"}`
`echo "1 - Value of var is ${var}"` `echo`
`${var:="Variable is not set"}` `echo "2`
`- Value of var is ${var}"` `unset var`
 - `echo ${var:+"This is default value"}`
`echo "3 - Value of var is $var"`
`var="Prefix"`
 - `echo ${var:+"This is default value"}`
`echo "4 - Value of var is $var"`
 - `echo ${var:?}"Print this message"}`
 - `echo "5 - Value of var is ${var}"`
- Upon execution, you will receive the following result –
 - Variable is not set
 - 1 - Value of var is Variable is not set
 - 2 - Value of var is Variable is not set
 - 3 - Value of var is This is default value
 - 4 - Value of var is Prefix
 - 5 - Value of var is Prefix

Shell Quoting Mechanisms

- Unix Shell provides various metacharacters which have special meaning while using them in any Shell Script and causes termination of a word unless quoted.
- For example, ? matches with a single character while listing files in a directory and an * matches more than one character.
- Here is a list of most of the shell special characters (also called metacharacters) –

* ? [] ' " \ \$; & () | ^ < > new-line space tab

- A character may be quoted (i.e., made to stand for itself) by preceding it with a \.

Example

- Example shows how to print a * or a ? –
#!/bin/sh
echo Hello; Word
- Upon execution, you will receive the following result
Hello
./test.sh: line 2: Word: command not found
shell returned 127
- Let us now try using a quoted character –
#!/bin/sh
echo Hello\; Word
- Upon execution, you will receive the following result
Hello; Word
- The \$ sign is one of the metacharacters, so it must be quoted to avoid special handling by the shell
#!/bin/sh
echo "I have \$1200"
- Upon execution, you will receive the following result
I have \$1200

The following table lists the four forms of quoting

Quoting	Description
Single quote	All special characters between these quotes lose their special meaning.
Double quote	Most special characters between these quotes lose their special meaning with these exceptions : \$, \\$ \' \" \\
Backslash	Any character immediately following the backslash loses its special meaning.
Back quote	Anything in between back quotes would be treated as a command and would be

The Single Quotes

- Consider an echo command that contains many special shell characters
`echo <-$1500.**>; (update?) [y|n]`
- Putting a backslash in front of each special character is tedious and makes the line difficult to read –
`echo \<-\$1500.\*\*\>\;; \ (update\?\) \ [y\|n\]`
- There is an easy way to quote a large group of characters. Put a single quote (') at the beginning and at the end of the string –
`echo '<-$1500.**>; (update?) [y|n]'`
- Characters within single quotes are quoted just as if a backslash is in front of each character. With this, the echo command displays in a proper way.
- If a single quote appears within a string to be output, you should not put the whole string within single quotes instead you should precede that using a backslash (\) as follows
`echo 'It\'s Shell Programming'`

The Double Quotes

- Try to execute the following shell script. This shell script makes use of single quote
VAR=ZARA
echo '\$VAR owes <-\$1500.**>; [as of `date +%m/%d`]'
- Upon execution, you will receive the following result –
\$VAR owes <-\$1500.**>; [as of `date +%m/%d`]
- This is not what had to be displayed. It is obvious that single quotes prevent variable substitution.
- If you want to substitute variable values and to make inverted commas work as expected, then you would need to put your commands in double quotes as follows –
VAR=ZARA
echo "\$VAR owes <-\ \$1500.**>; [as of `date +%m/%d`]"
- Upon execution, you will receive the following result –
ZARA owes <-\$1500.**>; [as of (07/02)]

The Double Quotes

- Double quotes take away the special meaning of all characters except the following –
 - \$ for parameter substitution
 - Backquotes for command substitution
 - \\$ to enable literal dollar signs
 - \` to enable literal backquotes
 - \" to enable embedded double quotes
 - \\ to enable embedded backslashes
- Characters within single quotes are quoted just as if a backslash is in front of each character. This helps the echo command display properly.
- If a single quote appears within a string to be output, you should not put the whole string within single quotes instead you should precede that using a backslash (\) as follows –
echo 'It\'s Shell Programming'

The Backquotes

- Putting any Shell command in between backquotes executes the command.
- The simple syntax to put any Shell command in between backquotes –
`var=`command``
- Example: The date command is executed in the following example and the produced result is stored in DATA variable.
`DATE=`date`
echo "Current Date: $DATE"`
- Upon execution, you will receive the following result –
Current Date: Thu Jul 2 05:28:45 MST 2009

Shell Functions

- Functions allows you to break down the overall functionality of a script into smaller, logical subsections, which can then be called upon to perform their individual tasks when needed.
- Using functions to perform repetitive tasks is an excellent way to create **code reuse**. This is an important part of modern object-oriented programming principles.
- Shell functions are similar to subroutines, procedures, and functions in other programming languages.
- To declare a function, simply use the following syntax

```
function_name () {  
    list of commands  
}
```
- The name of your function is `function_name`, and that's what you will use to call it from elsewhere in your scripts. The function name must be followed by parentheses, followed by a list of commands enclosed within braces.

Shell Functions: Example

- Following example shows the use of function –
#!/bin/sh

Define your function here Hello () {
 echo "Hello World"
}

Invoke your function
Hello
- Upon execution, you will receive the following output –
\$./test.sh
Hello World

Pass Parameters to a Function

- You can define a function that will accept parameters while calling the function. These parameters would be represented by \$1, \$2 and so on.
- Following is an example where we pass two parameters Zara and Ali and then we capture and print these parameters in the function.

```
#!/bin/sh
```

```
# Define your function here Hello () {  
    echo "Hello World $1 $2"  
}
```

```
# Invoke your function Hello Zara Ali
```

- Upon execution, you will receive the following result –

```
./test.sh
```

```
Hello World Zara Ali
```

Returning Values from Functions

- If you execute an exit command from inside a function, its effect is not only to terminate execution of the function but also of the shell program that called the function. If you instead want to just terminate execution of the function, then there is way to come out of a defined function.
- Based on the situation you can return any value from your function using the return command return code

- Following function returns a value 10 –

```
#!/bin/sh
```

```
# Define your function here Hello () {  
  echo "Hello World $1 $2" return 10  
}
```

```
# Invoke your function Hello Zara Ali
```

```
# Capture value returned by last command ret=$?  
echo "Return value is $ret"
```

- Upon execution, you will receive the following result –

```
./test.sh
```

```
Hello World Zara Ali Return value is 10
```

Nested Functions

- One of the more interesting features of functions is that they can call themselves and also other functions.
- A function that calls itself is known as a *recursive function*. For example

```
#!/bin/sh
# Calling one function from another number_one () {
    echo "This is the first function speaking..."
    number_two
}
number_two () {
    echo "This is now the second function speaking..."
}
# Calling function one. number_one
```

- Upon execution, you will receive the following result –
This is the first function speaking...
This is now the second function speaking...

Function Call from Prompt

- You can put definitions for commonly used functions inside your .profile. These definitions will be available whenever you log in and you can use them at the command prompt.
- Alternatively, you can group the definitions in a file, say test.sh, and then execute the file in the current shell by typing –
`$. test.sh`
- This has the effect of causing functions defined inside test.sh to be read and defined to the current shell as follows –
`$ number_one`
This is the first function speaking...
This is now the second function speaking...
`$`
- To remove the definition of a function from the shell, use the unset command with the .f option. This command is also used to remove the definition of a variable to the shell.
`$ unset -f function_name`